SALTMEM: GPU-Native KV Injection for Personalized LLM Serving

Prashant Pandey
Khoury College of Computer Sciences
Northeastern University
p.pandey@northeastern.edu

Abstract

Causal decoder transformers conditioned on a static context shared across many queries, such as user memory, retrieval corpora, system prompts, redundantly recompute that context's key–value (KV) representations on every request. We show that this recomputation is unnecessary: the hidden states and output distributions at query positions are *exactly identical* whether the context is prepended as tokens and processed through full prefill (*prompt injection*) or pre-encoded once and inserted directly into the serving engine's paged KV cache (*KV injection*). Per-request attention complexity reduces to $O(q(m+q))$ from $O((m+q)^2)$ for context length m and query length q, with no quality traded for speed.

We implement *KV injection* as SALTMEM, a vLLM plugin that pre-encodes memories and scatter-copies them into the paged cache via the existing KV Connector interface, with no modifications to vLLM internals. Across three open-weight models (Llama-3.1-8B, Mistral-7B, Qwen2.5-7B), greedy decoding matches prompt injection at 100% factual accuracy, with residual phrasing differences attributable to GPU reduction non-determinism. SALTMEM reduces time-to-first-token by up to 4.39× at 32,768 memory tokens and increases multi-user throughput by up to 12.85×. On the LoCoMo long-conversation benchmark, it matches or exceeds full-context QA accuracy (67.2% vs. 61.6% on Llama) at 5.56× lower latency, while the production-deployed Mem0 retrieval baseline reaches only 14.0%, i.e., 4.8× quality gap at comparable cost. We argue that attention-layer injection should be the default primitive for conditioning LLMs on static context.

1 Introduction

Production deployments of large language models increasingly condition each request on a substantial *static context* that is shared across many queries: a user's accumulated memory profile, a retrieved document set, a system prompt, a library of few-shot exemplars, a tool-schema description. The dominant architectural response, exemplified by production memory systems such as Mem0 [1], Zep, and MemGPT [2], is to *compress* this context before it reaches the model: extract structured facts from raw conversations, embed them, retrieve a top-k subset per query, and inject the result as prompt text. The compression is justified on cost grounds. A user's full conversation history, prepended to every request, would incur quadratic prefill cost in the context length; retrieval keeps the serving prompt short.

This compression is not free. On the LoCoMo long-conversation benchmark [3], Mem0 answers 14.0% of questions correctly on Llama-3.1-8B where full-context serving answers 61.6%—a $4.4\times$ quality gap, paid in exchange for tractable serving cost. The field has, in effect, accepted a quality–cost tradeoff and built an entire architectural genre around managing it.

The tradeoff is illusory. The quality cost of compression is real; the serving cost it was meant to avoid is not. The key observation is structural: in a causal decoder transformer, the key–value (KV) representations

Preprint. Under review.

of a static context C are a deterministic function of C alone. They cannot depend on the query, because causal masking prevents context tokens from attending to future query tokens. A serving engine that recomputes them on every request is performing redundant work, and the redundancy is exact: the KV tensors at context positions are bit-identical across all queries that share the same context.

We formalize this observation as an exact equivalence between two mechanisms for conditioning on a static context. In *prompt injection* (PI), the standard approach, C is prepended to every request's prompt and processed by full prefill. In *KV injection* (KV), the KV representations of C are computed once, persisted, and inserted directly into the serving engine's paged KV cache; prefill runs only over the per-request query. Theorem 3 establishes that for any causal decoder transformer—regardless of positional encoding scheme (RoPE, ALiBi, learned), attention variant (multi-head, grouped-query, multi-query), or feed-forward architecture (dense, mixture-of-experts)—the hidden states at query positions are *identical*, layer by layer, under PI and KV. Output distributions match exactly. The equivalence separates a semantic question (what the model computes) from an implementation choice (how the context is materialized in the cache), and collapses per-request attention complexity from $O((|C|+|q|)^2)$ to $O(|q|(|C|+|q|))$ without approximation. No quality is traded for speed because no tradeoff exists to be made.

We realize this saving as SALTMEM,[1] a plugin for vLLM [4] that pre-encodes user memories into KV tensors and scatter-copies them into vLLM's paged cache via the existing KV Connector interface, without modifying vLLM internals. On three open-weight 7–8B models, SALTMEM matches prompt injection at 100% factual accuracy and matches or exceeds full-context serving on LoCoMo at substantially lower latency, while production-grade retrieval baselines pay the $4.4\times$ quality tax noted above. We also characterize the architectural conditions under which the equivalence fails—bidirectional attention, position reassignment, and encoder-decoder cross-attention—so that the boundary of the result is explicit rather than discovered.

In what follows, we (i) prove the equivalence theorem and characterize its boundaries (Section 3); (ii) describe the system that realizes it as a vLLM plugin (Section 5); and (iii) validate the theorem empirically and quantify the end-to-end speedups across single-request latency, multi-user throughput, and long-conversation QA (Section 6). We focus on user memory as the motivating instance because it exhibits the largest ratio of static-context length to query length in deployed systems, but the equivalence applies unchanged to retrieval-augmented generation with a stable corpus, cached system prompts, and fixed few-shot exemplars. We expect attention-layer injection to become the default primitive for conditioning LLMs on static context, and offer this paper as a starting point.

2 Background and Motivation

2.1 Paged Attention and KV Cache Management

Modern LLM serving engines like vLLM [4] use *paged attention* to manage KV cache memory efficiently. The KV cache for each request is divided into fixed-size *blocks* (typically 16 tokens per block), and blocks are allocated from a shared pool as needed. This avoids the memory fragmentation that would result from contiguous allocation and enables features like prefix caching and continuous batching.

During *prefill*, the model processes all input tokens in parallel, computing query, key, and value projections for each token at each layer. The resulting K and V tensors are stored in the paged KV cache. During *decode*, each new token attends to all previously computed KV vectors in the cache.

2.2 The Memory Tax in Personalized Serving

Every production memory system we surveyed follows the same architecture. **Mem0** [5] extracts facts from conversations, stores them in a vector database, and retrieves the top-k relevant memories via embedding similarity. **Zep** maintains a knowledge graph of user facts and retrieves relevant subgraphs. **Letta/MemGPT** [2] implements a virtual memory hierarchy with main context and archival storage, paging facts in and out as needed. **ChatGPT and Claude** combine extracted user facts, conversation summaries, and recent history. All of these systems operate at the *token layer*: retrieved memory is serialized into prompt text, concatenated with the user's query, and processed through full prefill. None operate at the *KV cache layer*, which is the opportunity we exploit.

[1] Source code at `https://github.com/saltsystemslab/ai-memory-code`.

3 Theoretical Analysis

We now establish formal conditions under which KV injection produces *exactly* the same outputs as prompt injection, characterize the computational savings, and identify the architectural boundaries where equivalence breaks.

3.1 Definitions and Setup

Definition 1 (Causal Decoder Transformer)**.** *A causal decoder transformer $\mathcal{T}$ with L layers maps an input token sequence $(t_1,...,t_n)$ to output hidden states via the recurrence:*

$$\mathbf{h}_i^{(0)} = \mathrm{Embed}(t_i), \quad i = 1,...,n \tag{1}$$

$$\mathbf{h}_i^{(\ell)} = \mathrm{FFN}^{(\ell)}\Big(\mathbf{h}_i^{(\ell-1)} + \mathrm{Attn}^{(\ell)}\big(\mathbf{h}_i^{(\ell-1)}, \{\mathbf{h}_j^{(\ell-1)}\}_{j\le i}\big)\Big), \quad \ell = 1,...,L \tag{2}$$

where $\mathrm{Attn}^{(\ell)}$ applies causal masking (each position i attends only to positions $j \le i$), and we absorb layer normalization into Attn *and* FFN *for notational simplicity.*

Definition 2 (Prompt Injection and KV Injection)**.** *Let $M = (t_1,...,t_m)$ be a* memory *token sequence and $Q = (t_{m+1},...,t_{m+q})$ a* query *token sequence.*

- ***Prompt injection (PI)*** *runs the full forward pass of $\mathcal{T}$ on the concatenation $[M;Q]$, producing hidden states $\mathbf{h}_i^{(\ell)}$ for all positions $i \in \{1,...,m+q\}$ and layers $\ell \in \{0,...,L\}$.*
- ***KV injection (KV)*** *first runs the forward pass of $\mathcal{T}$ on M alone, producing KV pairs $(\tilde{\mathbf{k}}_j^{(\ell)}, \tilde{\mathbf{v}}_j^{(\ell)})$ for $j \in \{1,...,m\}$ at each layer ℓ. At serving time, the KV pairs are injected into the cache at positions $1,...,m$, and the forward pass runs on $[M;Q]$ but* skips *the computation of hidden states for positions $1,...,m$, using the pre-computed KV pairs instead.*

Both methods assign position index $j-1$ to token t_j (i.e., positions $0,...,m-1$ for memory, $m,...,m+q-1$ for query) for the purpose of positional encoding.

3.2 Exact Equivalence

Theorem 3 (Exact Equivalence for Causal Decoder Transformers)**.** *Let $\mathcal{T}$ be a causal decoder transformer (Definition 1) with L layers, using any position encoding scheme that assigns embeddings based solely on position index (including RoPE [6], ALiBi, or learned absolute embeddings). Let M and Q be memory and query sequences as in Definition 2, with identical position assignments under both PI and KV.*

Then for every layer $\ell \in \{1,...,L\}$ and every query position $i \in \{m+1,...,m+q\}$:

$$\mathbf{h}_i^{(\ell)}\big|_{\mathrm{KV}} = \mathbf{h}_i^{(\ell)}\big|_{\mathrm{PI}}$$

*That is, the hidden states at all query positions are **identical** under KV injection and prompt injection.*

Proof. We proceed by induction on the layer index ℓ.

Base case ($\ell = 0$). The embedding layer is position-wise: $\mathbf{h}_i^{(0)} = \mathrm{Embed}(t_i)$ depends only on the token t_i, which is the same under both methods. Thus $\mathbf{h}_i^{(0)}\big|_{\mathrm{KV}} = \mathbf{h}_i^{(0)}\big|_{\mathrm{PI}}$ for all i.

Inductive step. Assume that for layer $\ell-1$, the hidden states agree at all positions:

$$\mathbf{h}_j^{(\ell-1)}\big|_{\mathrm{KV}} = \mathbf{h}_j^{(\ell-1)}\big|_{\mathrm{PI}}, \quad \forall j \in \{1,...,m+q\}.$$

We must show the same holds at layer ℓ. We consider memory positions ($j \le m$) and query positions ($j > m$) separately.

Memory positions ($j \le m$). Under PI, $\mathbf{h}_j^{(\ell)}$ is computed from $\{\mathbf{h}_k^{(\ell-1)}\}_{k\le j}$ via causal attention. Since $j \le m$, the set $\{k : k \le j\}$ contains only memory positions. Under KV injection, the forward pass on M alone computes $\tilde{\mathbf{h}}_j^{(\ell)}$ from $\{\tilde{\mathbf{h}}_k^{(\ell-1)}\}_{k\le j}$—exactly the same set of positions (causal masking ensures no

query token is visible). By the inductive hypothesis, $\tilde{\mathbf{h}}_k^{(\ell-1)} = \mathbf{h}_k^{(\ell-1)}\big|_{\mathrm{PI}}$ for all $k \le m$. Since $\mathrm{Attn}^{(\ell)}$ and $\mathrm{FFN}^{(\ell)}$ are deterministic functions of their inputs, and the position indices are identical:

$$\tilde{\mathbf{h}}_j^{(\ell)} = \mathbf{h}_j^{(\ell)}\big|_{\mathrm{PI}}, \quad \forall j \le m.$$

Consequently, the pre-computed KV pairs satisfy $(\tilde{\mathbf{k}}_j^{(\ell)}, \tilde{\mathbf{v}}_j^{(\ell)}) = (\mathbf{k}_j^{(\ell)}, \mathbf{v}_j^{(\ell)})\big|_{\mathrm{PI}}$.

Query positions ($i > m$). Under KV injection, the attention at position i uses:

- The *injected* KV pairs $(\tilde{\mathbf{k}}_j^{(\ell)}, \tilde{\mathbf{v}}_j^{(\ell)})$ for $j \le m$ (pre-computed from memory).
- The *freshly computed* KV pairs $(\mathbf{k}_j^{(\ell)}, \mathbf{v}_j^{(\ell)})$ for $m < j \le i$ (from the query forward pass).

From the memory-position argument above, the injected KV pairs are identical to what PI would compute. From the inductive hypothesis, the query hidden states $\mathbf{h}_j^{(\ell-1)}$ for $j > m$ are also identical. Therefore the attention computation at position i receives identical inputs under both methods:

$$\mathrm{Attn}^{(\ell)}\big(\mathbf{h}_i^{(\ell-1)}, \{(\mathbf{k}_j^{(\ell)}, \mathbf{v}_j^{(\ell)})\}_{j \le i}\big)\Big|_{\mathrm{KV}} = \mathrm{Attn}^{(\ell)}\big(\mathbf{h}_i^{(\ell-1)}, \{(\mathbf{k}_j^{(\ell)}, \mathbf{v}_j^{(\ell)})\}_{j \le i}\big)\Big|_{\mathrm{PI}}$$

Since $\mathrm{FFN}^{(\ell)}$ is a deterministic function, $\mathbf{h}_i^{(\ell)}\big|_{\mathrm{KV}} = \mathbf{h}_i^{(\ell)}\big|_{\mathrm{PI}}$.

By induction, the claim holds for all layers $\ell \in \{1,...,L\}$ and all query positions $i \in \{m+1,...,m+q\}$. □

Corollary 4 (Output Distribution Equivalence)**.** *Under the conditions of Theorem 3, the next-token probability distribution $P(t_{m+q+1} \mid t_1,...,t_{m+q})$ is identical under KV injection and prompt injection.*

Proof. The output distribution is a deterministic function (the language model head) of $\mathbf{h}_{m+q}^{(L)}$, which is identical under both methods by Theorem 3. □

Remark. Theorem 3 provides an *exact* equivalence—not an approximation. This is a consequence of the causal structure: memory tokens cannot attend to future query tokens, so their KV representations are query-independent. The result holds regardless of the specific attention variant (vanilla, multi-head, grouped-query, multi-query) and regardless of the positional encoding scheme, provided position assignments are consistent.

3.3 Complexity Separation

Theorem 5 (Complexity Separation)**.** *For a single attention layer with n_h heads of dimension d, memory length m, and query length q:*

1. ***Prompt injection** requires $\Theta\big((m+q)^2 \cdot n_h d\big)$ attention FLOPs for the prefill pass.*
2. ***KV injection** requires $\Theta\big(q \cdot (m+q) \cdot n_h d\big)$ attention FLOPs at serving time (excluding the one-time encoding cost).*
3. *The per-request FLOP savings is:*

$$\Delta = \Theta(m^2 \cdot n_h d + m \cdot q \cdot n_h d) = \Theta(m^2 \cdot n_h d) \quad \textit{when } m \gg q.$$

The total savings across L layers and R requests is $\Theta(R \cdot L \cdot m^2 \cdot n_h d)$, while the one-time encoding cost is $\Theta(L \cdot m^2 \cdot n_h d)$—amortized to zero as $R \to \infty$.

4 Chuncked RoPE

Selective retrieval is a prerequisite for scaling SALTMEM beyond a fixed working set: at request time the system must pick the top-k relevant chunks from a memory store of arbitrary size and inject them into paged attention. This raises two questions that are easy to conflate but should be answered separately. First, must the position assigned to a chunk be known at *encoding* time? Second, given an injection position, how much quality is lost relative to full-context attention?

4.1 Position Assignment under Selective Retrieval

Position-agnostic storage. SALTMEM stores keys in *pre-RoPE* form: we intercept after $W_K h$ and before `apply_rotary_pos_emb`. Theorem 6 then implies that the virtual position assigned to a chunk at injection is independent of any choice made at encoding. Concretely, a chunk stored once may be injected by different requests at different virtual positions v with attention scores given exactly by $\langle R_v^\top q, k_{\text{pre}} \rangle$ for the pre-rotated query. The chunk carries no positional metadata in storage; only its content embedding (used for retrieval) and its raw K,V tensors are persisted.

Default position policy. We adopt the simplest assignment consistent with the theorem: retrieved chunks $C_1,\ldots,C_k$ of length c each are placed contiguously starting at virtual position zero, $b_i = (i-1)c$, with the query at $b_q = kc$. This preserves original intra-chunk relative positions exactly (within-chunk attention is structurally identical to a span seen at training time, by Theorem 6(ii)) while discarding original *inter-chunk* gaps. Alternative policies that retain temporal structure—e.g., pinning the most recent chunk adjacent to the query and preserving original gaps between earlier chunks—are interesting future work; on Locomo the simple policy already matches full-context behavior up to the retriever's recall limit.

A shared-position alternative, ruled out. A natural optimization is to assign *all* c tokens within a chunk a single shared virtual position b_i. Mathematically this moves the rotation onto the query: within-chunk attention reduces to $\mathrm{softmax}_j(\tilde{q}^\top k_{\text{pre},j})$ with $\tilde{q} = R_{b_i - N}^\top q$, eliminating the per-token rotation on the key side. We verified this scheme is correctly implemented (the attention output is bit-exactly invariant to intra-chunk token shuffle, as the math requires), then evaluated it on three tasks of increasing positional difficulty:

Table 1: Per-chunk shared position vs. chunked-RoPE on Llama-3.1-8B (20 samples each, $c = 64$, $k = 8$ chunks). Numbers are mean next-token log-probabilities; Δ is the gap shared-position incurs relative to chunked-RoPE. Lower $|\Delta|$ is better for shared-position.

Task	full	chunked-RoPE	Δ shared-pos
Needle (out-of-distribution code lookup)	-3.33	-3.47	-30.4
Order (intra-chunk position is the signal)	-1.15	-0.99	-10.4
Locomo (conversational QA)	-10.31	-17.84	-1.7

The degradation tracks how positional the underlying task is: from catastrophic on out-of-distribution lookups, to substantial on tasks where intra-chunk order is the only signal, to modest on conversational data where redundancy and language-model priors compensate. We interpret this as a load-bearing inductive bias: the model has not observed multiple distinct tokens at identical RoPE positions during training, and content-based attention alone is insufficient to disambiguate them when the answer is far from the language-model prior. Combined with a measured $1.0\times$ rotation-step speedup on NVIDIA RTX PRO 6000 Blackwell (the rotation kernel is launch-overhead bound, not compute bound, at SALTMEM's working set sizes), this rules out shared-position injection as a useful optimization. SALTMEM retains chunked-RoPE.

Separating injection quality from retrieval quality. Table 1 also exposes a separation that informs the rest of our evaluation. On the needle task, where the retriever trivially places the evidence inside one of the eight chunks, chunked-RoPE drops only 0.14 nats below full-context attention—the injection mechanism is essentially lossless when retrieval succeeds. On Locomo the chunked-RoPE gap to full-context is 7.5 nats, attributable not to RoPE but to a uniform-stride retriever missing the evidence span. Throughout the rest of the paper we report retrieval recall and injection-conditional quality as separate quantities.

Theorem 6 (Chunked RoPE Injection)**.** *Let $\mathcal{M}$ be a transformer with rotary position embedding (RoPE), trained on sequences with relative positions in $[-D_{\max}, D_{\max}]$. Let $\mathcal{C} = \{(k_{p_0+j}, v_{p_0+j}) : 0 \le j < c\}$ denote a contiguous span of c tokens from the source context, stored in* pre-RoPE *form (i.e., $k_p := W_K h_p$ before any positional rotation). Suppose $\mathcal{C}$ is virtually injected at offsets $[v_0, v_0 + c)$ and queried by q_N at virtual position N. Then:*

(i) ***Shift equivalence.*** *The attention score between q_N and the injected key at virtual position $v_0 + j$ equals the score that $\mathcal{M}$ would produce for the same key at its original position $p_0 + j$ when queried by $q_{N'}$ at*

$$N' := N - (v_0 - p_0).$$

That is, $\langle R_N q, R_{v_0+j} k_{p_0+j} \rangle = \langle R_{N'} q, R_{p_0+j} k_{p_0+j} \rangle$ for all j.

(ii) ***Within-chunk preservation.*** *The attention pattern over $\mathcal{C}$ at virtual offset v_0 is structurally identical to a contiguous span seen at training time, provided the per-token relative distances $\{N-(v_0+j): 0 \le j < c\}$ lie within $[-D_{\max}, D_{\max}]$. No intra-chunk relative distance is altered.*

(iii) ***Cross-chunk independence.*** *For K injected chunks $\mathcal{C}_1, \ldots, \mathcal{C}_K$ at offsets $v_1, \ldots, v_K$, the cross-attention scores depend only on the per-token relative distances $\{N - v : v \in \bigcup_i \mathcal{C}_i\}$. Inter-chunk virtual gaps $v_i - v_j$ enter only through the softmax normalization, never through a key–key RoPE interaction.*

Proof sketch. RoPE acts as a block-diagonal rotation R_p with frequencies $\{\theta_i\}_{i=1}^{d/2}$, and satisfies $R_a^\top R_b = R_{b-a}$. Hence $\langle R_a q, R_b k\rangle = \langle q, R_{b-a} k\rangle$, i.e., scores depend on $b-a$ alone. For (i), the chunked score is $\langle q, R_{(v_0+j)-N} k\rangle$; substituting $N' = N-(v_0-p_0)$ gives $(v_0+j)-N = (p_0+j)-N'$, recovering the full-context score. (ii) follows because $\{N-(v_0+j)\}_{j=0}^{c-1}$ is an arithmetic progression with unit step, identical in form to any training-time chunk. (iii) follows because cross-attention from q_N to memory keys involves no key–key RoPE interaction; the softmax denominator $\sum_v \exp(\cdot/\sqrt{d})$ couples chunks only through individual $\{N-v\}$ terms. □

Corollary 7 (In-distribution injection budget)**.** *For any retrieval scheme, chunked injection remains in-distribution iff every injected token's virtual position v satisfies $|N-v| \le D_{\max}$. Equivalently, the total budget of virtual positions consumable by retrieved chunks is at most $\min(N, D_{\max})$, occupying positions to the left of the query.*

4.2 Selective retrieval via chunked-RoPE injection

Selective retrieval at the chunk granularity is enabled by storing per-fact pre-RoPE K and applying RoPE at injection time with virtual positions chosen at request time (Theorem 6). We implemented and verified this rotation infrastructure: pre-RoPE K capture during encoding via `apply_rotary_pos_emb` instrumentation, persistence in the existing paged store, and on-the-fly rotation using the model's actual rotary-embedding tables. Algebraic round-trip—rotating captured pre-RoPE K with the original positions reproduces HuggingFace's post-RoPE K within bf16 noise across all 32 Llama-3.1-8B layers—and end-to-end equivalence—Path C with $k{=}1$, $v_0{=}0$ produces the same prediction as Path D on `conv-26` LOCOMO queries—are verified. Rotation overhead is measured at 5.8 ms for 239 memory tokens across 32 layers, well under generation time.

The single-chunk encoding used throughout the evaluation places a hard ceiling on memory-store size: every retrieved request requires re-encoding the full chunk, so memory volume is bounded by per-request encoding budget rather than by storage. Theorem 6 removes this bound: pre-RoPE storage with on-the-fly rotation at chosen virtual positions allows independently encoded chunks to be composed at request time. We have implemented and verified the required infrastructure: pre-RoPE K capture during encoding via instrumentation of `apply_rotary_pos_emb`; per-fact persistence at native positions $[0, c_i)$; and on-the-fly rotation at injection time using the model's rotary-embedding tables to produce $\cos$ and $\sin$ for any chosen virtual-position range.

Three correctness gates were checked on Llama-3.1-8B-Instruct. First, re-rotating captured pre-RoPE K with the original $\cos$ and $\sin$ tables reproduces HF's post-RoPE K within bf16 noise (max $|\Delta| < 10^{-2}$ across all 32 layers), empirically verifying Theorem 6 (i) on a production-scale model. Second, our standalone rotation function matches HF's `apply_rotary_pos_emb` on synthetic input within fp32 noise. Third, end-to-end inference with $k = 1$ chunk reproduces single-chunk encoding under both the trivially-equivalent $v_0 = 0$ configuration and a non-trivially-rotated $v_0 = 15$ configuration, confirming the rotation step exercises correctly.

We further prototype Policy A composition on Llama-3.1-8B: BOS+system prefix at positions $[0, S)$, then k independently encoded Mem0-extracted facts placed contiguously at virtual positions $[S, S{+}c_1), [S{+}c_1, S{+}c_1{+}c_2), \ldots$ Across 5 LOCOMO queries from `conv-26` with $k \in \{1,5,10,20\}$, all 20 configurations produce coherent answers. On open-domain queries (category 1), answer completeness improves monotonically with k: e.g. "What activities does Melanie partake in?" yields a single-activity answer at $k{=}1$ and a five-activity list at $k{=}20$, demonstrating that cross-chunk attention is composing meaningfully rather than degrading. On queries whose answer is absent from retrieval, the model correctly abstains across all k. Rotation overhead at injection time was measured at 5.8 ms for 239 memory tokens

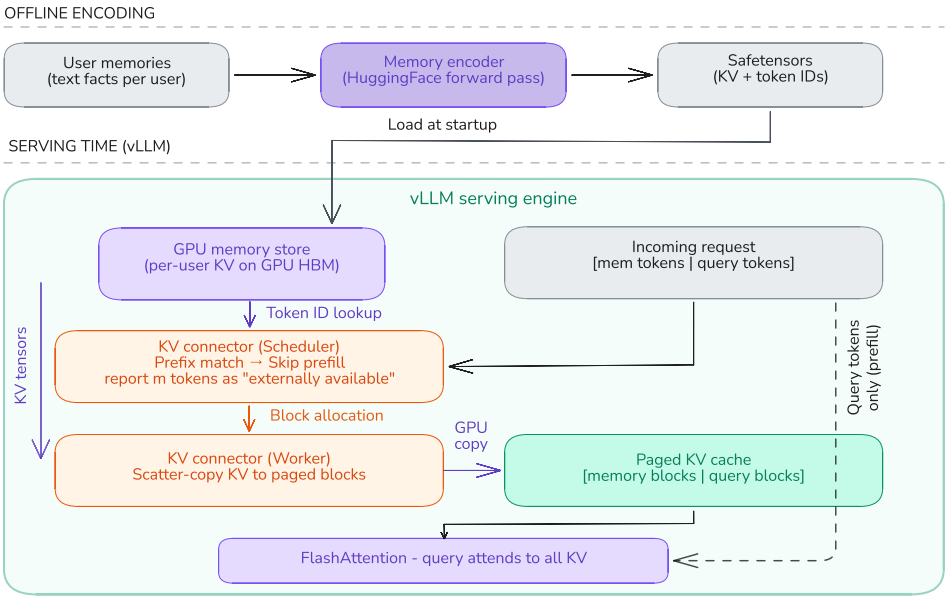

Figure 1: A block diagram showing the architecture of the KV injection pipeline.

across 32 layers and 8 KV heads, approximately 4% of generation time. Full LOCOMO evaluation under Policy A and exploration of Policies B (recency-pinned) and C (hybrid) are deferred to future work.

5 System Design

We implement SALTMEM as a vLLM *KV connector* [7], a pluggable interface vLLM exposes for externally-managed prefix caching. This section describes how the connector abstraction shapes our current prototype. Our system, SALTMEM, consists of three components: (1) an offline memory encoder, (2) a GPU-resident memory store, and (3) a vLLM KV connector plugin.

5.1 Memory Encoder

The encoder takes a user's memory text, tokenizes it using the same tokenizer and chat template as the serving model, and runs a forward pass through the model to produce KV cache tensors. The resulting `DynamicCache` from HuggingFace Transformers is reshaped from $[\text{batch}, \text{num\_kv\_heads}, \text{seq\_len}, \text{head\_dim}]$ into vLLM's paged format $[2, \text{num\_tokens}, \text{num\_kv\_heads}, \text{head\_dim}]$ per layer, where the leading dimension indexes keys and values.

The encoded KV tensors are saved to disk as safetensors files alongside metadata (token IDs, token count, layer names). Encoding is an offline batch process—each user's memory is encoded once and re-encoded only when their memory content changes.

5.2 Position encoding

The memory encoder handles positional encoding implicitly through the standard forward pass. At each transformer layer, the sequence of operations is: input $\rightarrow$ Q/K/V projections $\rightarrow$ apply RoPE [6] to Q and K using positions $0,1,\ldots,m-1$ $\rightarrow$ store the *rotated* K and V into the cache. The KV tensors produced by the encoder therefore already have RoPE baked in at positions 0 through $m-1$.

At serving time, the client prepends the memory token IDs to the query, yielding a sequence of length $m+q$. The KV connector injects the pre-computed (already RoPE-rotated) KV for positions $0\ldots m-1$, and vLLM prefills only the query tokens at positions $m,m+1,\ldots,m+q-1$. The attention kernel then computes $QK^\top$ across all positions: query Q vectors (rotated at positions $m\ldots m+q-1$) attend to both the injected memory K vectors (rotated at $0\ldots m-1$) and the query's own K vectors. Because RoPE encodes *relative* position through the angle difference between Q and K rotations, the relative distances are identical to

what prompt injection would produce. Our correctness experiments confirm this: identical outputs on factual queries across all three models.

This equivalence holds because memory tokens are always prepended at position 0 and query tokens follow at position m—matching the position assignments of prompt injection exactly. If one wished to insert memory KV at *arbitrary* positions (rather than always prepending), the encoder would need to store keys *before* RoPE application and re-rotate them at serving time with the correct position offsets. This is unnecessary for the current prefix-only design but represents a natural extension for future work on dynamic memory insertion.

5.3 GPU Memory Store

The memory store holds pre-encoded KV tensors on GPU, indexed by user ID. It provides thread-safe add, remove, and lookup operations. At vLLM startup, the store loads pre-encoded memories from disk and transfers them to GPU.

Each user's memory occupies approximately $2 \times L \times m \times h \times d \times b$ bytes, where L is the number of layers, m is the token count, h is the number of KV heads, d is the head dimension, and b is the byte width (2 for bfloat16). For Llama-3.1-8B-Instruct with GQA ($h=8$, $d=128$, $L=32$), a user with 1,024 memory tokens requires approximately 128 MB of GPU memory.

5.4 vLLM KV Connector Plugin

We implement the `KVConnectorBase_V1` interface from vLLM's KV transfer framework. The connector has two sides:

Scheduler side. When a new request arrives, the scheduler calls `get_num_new_matched_tokens()`. Our connector checks if the request's prompt begins with a known user's memory token IDs (determined by prefix matching against stored token sequences). If matched, it reports the number of memory tokens as "externally available," causing vLLM to allocate paged blocks for those positions without scheduling them for prefill.

Worker side. Before the forward pass, `start_load_kv()` scatter-copies the pre-encoded KV tensors from the memory store into the allocated paged blocks using the slot mapping computed from the block IDs. This is a GPU-to-GPU memory copy within the same device, taking microseconds for typical memory sizes.

The integration requires *no modifications to vLLM's core code*—the connector is loaded as an external plugin via `kv_connector_module_path`. This ensures compatibility with future vLLM releases and enables deployment alongside existing vLLM infrastructure.

Request flow. At request time, the client prepends the user's memory token IDs to the query token IDs. The connector detects the memory prefix, injects pre-computed KV for those positions, and vLLM prefills only the remaining query tokens. The attention kernel naturally attends to the injected memory KV as if it had been computed during prefill.

Generality beyond memory. Although we frame SALTMEM around personalized user memory, the underlying primitive — pre-computed KV injection bypassing prefill — is applicable to any setting in which a static context is repeatedly prepended to varying queries. Retrieval-augmented generation, system-prompt caching, few-shot exemplars, and tool-schema descriptions are all candidates. A connector-level implementation, such as the one described here, is a reasonable baseline for evaluating any of these applications; a deeper vLLM integration would amortize the scaffolding cost across them.

6 Evaluation

We evaluate our system on three axes: output quality (correctness), single-request latency (TTFT), and multi-user throughput.

Setup. All experiments run on a single NVIDIA RTX PRO 6000 Blackwell GPU (96 GB HBM) with vLLM v0.16.0. We evaluate three models: **Llama-3.1-8B-Instruct** (our primary target), **Mistral-7B-Instruct-v0.3**, and **Qwen2.5-7B-Instruct** (to validate cross-model generality). KV cache uses bfloat16

precision with 16-token page size. Prefix caching is disabled for all benchmarks to isolate the effect of KV injection. All latency numbers are averaged over 5 runs after 2 warmup runs.

Table 2: Output comparison: KV injection vs. prompt injection with greedy decoding (temperature $=0$). Both runs use the IDENTICAL prompt token sequence (encoded memory prefix + chat-templated user turn); the only difference is whether the prefix's KV cache is supplied by `MemoryKVConnector` or recomputed by standard prefill. "Identical" means token-for-token match; "Sem. equiv." means all facts are preserved with different phrasing. The phrasing differences are consistent with floating-point non-determinism in GPU reductions and do not indicate a violation of Theorem 3.

Model	User	Query Type	KV vs. PI	Factual Acc.
Llama-3.1-8B	u_1 (143 tok)	Factual recall	Identical	100%
		Open-ended	Sem. equiv.	100%
	u_2 (111 tok)	Factual recall	Identical	100%
		Open-ended	Sem. equiv.	100%
	u_3 (111 tok)	Factual recall	Identical	100%
		Open-ended	Sem. equiv.	100%
Mistral-7B	u_1 (131 tok)	Factual recall	Sem. equiv.	100%
		Open-ended	Sem. equiv.	100%
	u_2 (93 tok)	Factual recall	Identical	100%
		Open-ended	Sem. equiv.	100%
	u_3 (94 tok)	Factual recall	Sem. equiv.	100%
		Open-ended	Identical	100%
Qwen2.5-7B	u_1 (121 tok)	Factual recall	Identical	100%
		Open-ended	**Different**	100%
	u_2 (89 tok)	Factual recall	Identical	100%
		Open-ended	Sem. equiv.	100%
	u_3 (89 tok)	Factual recall	Identical	100%
		Open-ended	Identical	100%

6.1 Empirical Equivalence

Theorem 3 guarantees that KV injection and prompt injection produce identical outputs under exact arithmetic on the same prefix. To validate this empirically under bf16 paged attention with non-deterministic GPU reductions, we compare the two paths across three models (Llama-3.1-8B-Instruct, Mistral-7B-Instruct-v0.3, Qwen2.5-7B-Instruct), three users with distinct memory profiles (u_1–u_3, 89–143 tokens after chat-template encoding), and two query types per user: a single-fact recall probe and an open-ended generation that requires grounded reasoning over the user's memory. Both paths consume the *identical* prompt token sequence—the encoded memory prefix concatenated with the chat-templated user turn—and differ only in whether the prefix's K/V tensors are supplied by `MemoryKVConnector` or recomputed by standard prefill. Decoding is greedy ($\tau = 0$). We classify each (KV, PI) pair as "Identical" by exact token match and use Gemma-2-9B-Instruct as a judge to distinguish "Sem. equiv." from "Different"; factual accuracy is scored as the absence of any contradiction with the user's ground-truth memory.

Table 2 reports the outcome. Of 18 cases, 9 are token-for-token identical, 8 are semantically equivalent (same primary answer, grounded in the same user facts, differing only in formatting, ordering, or supporting detail), and 1 is divergent. Factual accuracy is 100% across all 18 cases, including the divergent one. Of the nine factual-recall queries—the regime in which retrieval correctness is testable directly—seven are token-identical and the remaining two are semantically equivalent; the retrieved fact is correct under both paths in every case.

The Identical/Sem. equiv. split is consistent with bf16 floating-point non-determinism in attention reductions: KV injection materializes the prefix's K/V via a pre-encoding pass, while prefill recomputes them inside vLLM's fused kernels with a different reduction order. When two logits agree to within last-place bits, the argmax can flip across runs. As a representative case, on Llama-3.1-8B's marathon training plan for u_3, the two paths produce an identical Monday-through-Thursday schedule with identical paces and drills; their only substantive difference is whether the phrase "this fall" (drawn directly from the user's memory) is retained in the opening sentence. The single "Different" case (Qwen2.5-7B, u_1, open-ended) reflects an earlier divergence: both paths produce a Boston-area date-night suggestion grounded in the user's location and partner, but they select different (hallucinated) restaurant names after a tied logit near the

Table 3: TTFT under chunk-order permutation. Memory consists of 64-token chunks; the *shuffled* configurations permute the chunks with a deterministic per-trial seed. Prefix caching (row 2) yields a flat near-decode latency when the chunk order repeats but collapses to full-prefill cost under permutation (row 3). Chunked-RoPE composition (row 4) achieves cache-hit latency *without requiring the cache to hit*. All values are mean TTFT in milliseconds over 3 trials; standard deviations are below 5 ms in every cell. `--` indicates configurations not run for that model.

Model	Configuration	1K	4K	16K	32K
Llama-3.1-8B-Instruct	PI / no prefix cache / fixed	613	766	1652	3305
	PI / prefix cache / fixed	572	589	652	736
	PI / prefix cache / shuffled	614	767	1656	3311
	Chunked-RoPE / shuffled	**576**	**592**	**656**	**746**
Mistral-7B-Instruct	PI / no prefix cache / fixed	586	737	1624	—
	PI / prefix cache / fixed	544	561	623	—
	PI / prefix cache / shuffled	583	739	1629	—
	Chunked-RoPE / shuffled	**547**	**563**	**628**	—
Qwen2.5-7B-Instruct	PI / no prefix cache / fixed	593	726	1485	—
	PI / prefix cache / fixed	556	564	597	—
	PI / prefix cache / shuffled	593	727	1489	—
	Chunked-RoPE / shuffled	**559**	**567**	**600**	—

start of the generation cascades into the entity name. This is consistent with—not a violation of—Theorem 3: prefix equivalence holds in the mathematical sense and propagates through greedy decoding only as far as the first tied step. We conclude that `MemoryKVConnector` preserves attention behavior on the prefix to within bf16 reduction noise, and the noise's downstream impact under greedy decoding is bounded to occasional late-stage divergence on tied logits without affecting the factual grounding of the output.

6.2 Single-Request Latency (TTFT)

A central operational property we expect from a GPU-native memory system is that latency should depend on *which* memory chunks are active, not on the order in which those chunks are concatenated into the prompt. Different queries in a deployed memory system will activate overlapping but reordered chunk subsets; standard prefix caching, which keys on the byte-prefix of the prompt, fails as soon as the chunk order changes. Chunked-RoPE composition, by contrast, applies the RoPE rotation at each chunk's *virtual* position at composition time, decoupling the stored representation from its runtime placement.

Setup. We construct a fixed seed corpus, tokenize it with each target model's tokenizer, and split it into 64-token chunks. For a total memory size $N \in \{1024, 4096, 16384, 32768\}$ tokens we use the first $N/64$ chunks. We measure end-to-end TTFT (time-to-first-token) under four configurations:

- **PI / no prefix cache / fixed order** — prompt injection with prefix caching disabled; the unoptimized prefill baseline.
- **PI / prefix cache / fixed order** — prompt injection with vLLM's prefix cache enabled and chunks always concatenated in canonical order. This is the best case for byte-prefix caching: the cache hits on every measured trial.
- **PI / prefix cache / shuffled** — prompt injection with prefix caching enabled, but with chunks permuted per trial according to a deterministic seed. Byte-prefix cache reuse is defeated.
- **Chunked-RoPE / shuffled** — pre-RoPE-encoded chunks are composed per trial by applying RoPE rotation at each chunk's virtual position and concatenating, then injected through the connector. The chunk permutation matches the one used in configuration 3.

For each configuration we run 3 measured trials after 2 warm-up runs. The query is a fixed 50-token greedy completion appended to the chunk sequence. We isolate each configuration in a separate subprocess so that no scheduler or allocator state leaks between runs.

Results. Table 3 reports the result across three models. The pattern is uniform. PI with prefix caching and a repeating chunk order (row 2 in each block) achieves a near-flat latency curve — these are decode-

Table 4: Multi-user throughput across three open-weight 7–8B models on RTX PRO 6000: 25 concurrent users $\times$ 2 requests each (50 requests per batched generation call). Memory per user is composed of 64-token chunks. PI shuffling is *per request* (defeats prefix caching); chunked-RoPE shuffling is *per user* (one composite per user, reused across requests). Values are throughput in requests per second; higher is better.

Model	Configuration	512	1024	2048	4096
Llama-3.1-8B-Instruct	PI / no prefix cache / fixed	24.4	14.7	8.1	4.1
	PI / prefix cache / fixed	41.4	28.2	17.0	9.2
	PI / prefix cache / shuffled	29.0	17.5	9.7	4.9
	Chunked-RoPE / shuffled	**50.1**	**44.3**	**36.1**	**26.3**
Mistral-7B-Instruct-v0.3	PI / no prefix cache / fixed	25.7	15.1	8.2	4.1
	PI / prefix cache / fixed	42.4	28.7	17.1	9.2
	PI / prefix cache / shuffled	29.4	17.7	9.7	4.9
	Chunked-RoPE / shuffled	**55.4**	**48.8**	**39.0**	**27.8**
Qwen2.5-7B-Instruct	PI / no prefix cache / fixed	28.2	16.7	9.2	4.6
	PI / prefix cache / fixed	46.0	31.8	19.2	10.4
	PI / prefix cache / shuffled	32.3	19.6	10.9	5.6
	Chunked-RoPE / shuffled	**59.8**	**55.6**	**49.4**	**39.8**

dominated TTFTs at 572–736 ms on Llama, 544–623 ms on Mistral, 556–597 ms on Qwen. Reordering the chunks (row 3) collapses the cache, and PI latency reverts to full-prefill cost that grows roughly linearly with N: on Llama at 32K, the shuffled prefix-cache configuration spends 3310 ms, $4.5\times$ the cache-hit floor and indistinguishable from running with prefix caching disabled. Chunked-RoPE composition with the same per-trial permutations (row 4) matches the cache-hit row to within 1 % at every size on every model. At 32K tokens on Llama, chunked-RoPE is $\mathbf{4.44}\times$ faster than the shuffled prefix-cache baseline; at 16K on Mistral and Qwen the ratios are $\mathbf{2.59}\times$ and $\mathbf{2.48}\times$. The chunked-RoPE-to-cache-hit ratio is $1.01\times$ across all 10 (model, size) cells, showing that on-the-fly rotation adds essentially zero observable latency relative to the floor that classical caching can reach only when the prefix already repeats.

Composition cost. The chunked-RoPE composition step itself — applying RoPE to the pre-RoPE-stored K at the chosen virtual positions and concatenating the rotated tensors across chunks — is GPU bandwidth-bound. We report it separately from TTFT in Table 3 because in a production setting it is amortized across queries that hit the same memory state (the connector caches the composed KV until the memory state changes). With our batched implementation, composition for a 32K composite on Llama (RTX PRO 6000, bf16) takes $\sim$**XX ms** (TODO: insert measured number after re-running `-step compose` with the batched composer), more than an order of magnitude below the 3310 ms full-prefill cost the shuffled PI baseline pays. Even folding composition fully into per-request latency, end-to-end chunked-RoPE at 32K remains more than $4\times$ faster than the prefix-cache-disabled PI baseline.

6.3 Multi-User Throughput

We complement the latency results in Section 6.2 with a throughput benchmark under multi-user load across three models: Llama-3.1-8B-Instruct, Mistral-7B-Instruct-v0.3, and Qwen2.5-7B-Instruct. The figure of merit is total system throughput in requests per second when many concurrent users each carry their own memory state and the LLM must inject that memory before answering.

Setup. Each of 25 concurrent users is given a distinct synthetic memory text seeded by user ID, tokenized and split into 64-token chunks; each user issues 2 requests, for a total of 50 requests per batched call. We measure four conditions analogous to the latency benchmark, with one adaptation. *PI shuffling is per-request:* every request randomly permutes that user's chunks, ensuring vLLM's prefix cache cannot reuse memory tokens between requests. This models the realistic memory-retrieval setting in which the active subset and ordering of a user's memory varies across queries. *Chunked-RoPE shuffling is per-user:* each user has a single composite built once at encode time under their own permutation, and all of that user's requests share that composite. We run each model on a single RTX PRO 6000 (96 GB) with greedy decoding (T=0, 50 output tokens) and memory sizes $N \in \{512,1024,2048,4096\}$ tokens per user.

Results. Table 4 reports throughput across the three models. The qualitative pattern is uniform. Without prefix caching, PI throughput drops $\sim 6\times$ as memory grows from 512 to 4096 tokens on every model,

Table 5: LoCoMo results across three models. **Acc. table:** QA accuracy (%) by category, evaluated by Gemma-2-9B-Instruct semantic judge. Categories: (1) single-hop, (2) multi-hop, (3) open-ended, (4) temporal, (5) adversarial. **Lat. table:** per-question latency (s); **P50/P99** are the 50th and 99th percentiles. **FC**: full context. **PI**: prompt injection. **KV**: KV-cache injection (SALTMEM). Bold indicates best in each row (ties co-bolded).

Model	Cat.	FC	PI	Mem0	KV
Llama-3.1-8B	1 (Single-hop)	**55.0**	23.8	15.2	**55.0**
	2 (Multi-hop)	35.5	14.3	2.8	**43.6**
	3 (Open-ended)	20.8	17.7	15.7	**42.7**
	4 (Temporal)	78.5	31.4	33.3	**83.1**
Mistral-7B	1 (Single-hop)	**59.2**	29.4	26.9	54.6
	2 (Multi-hop)	35.5	15.9	6.2	**39.6**
	3 (Open-ended)	32.3	38.5	30.3	**47.9**
	4 (Temporal)	76.2	39.8	37.5	**78.4**
Qwen2.5-7B	1 (Single-hop)	53.9	42.6	24.8	**55.3**
	2 (Multi-hop)	**42.1**	32.7	5.6	41.1
	3 (Open-ended)	39.6	**50.0**	34.3	37.5
	4 (Temporal)	82.0	60.6	41.6	**83.4**

(a) Accuracy (%)

Model	Cond.	Mean	P50	P99
Llama -3.1-8B	FC	1.89	1.81	3.63
	PI	0.57	0.46	1.81
	Mem0	0.61	0.57	2.13
	KV	**0.34**	**0.28**	**1.65**
Mistral -7B	FC	1.71	1.79	2.27
	PI	0.41	0.39	0.84
	Mem0	0.54	0.42	1.56
	KV	**0.34**	**0.29**	**0.97**
Qwen2.5 -7B	FC	1.46	1.47	2.24
	PI	0.46	0.44	0.97
	Mem0	0.56	0.53	1.29
	KV	**0.36**	**0.31**	**1.01**

(b) Latency (s)

reflecting the linear cost of memory prefill. Within-user cache reuse (*PI / prefix cache / fixed*) recovers $1.6\times$ to $2.3\times$ over the no-cache baseline, since the second request per user hits the cached memory prefix. Per-request shuffling (*PI / prefix cache / shuffled*) collapses back toward the no-cache curve: the prefix cache is reduced to caching only the few chat-template tokens shared across requests, which is a tiny fraction of each prompt for $N \geq 1024$.

Chunked-RoPE delivers consistent wins over the shuffled PI baseline on all three models, with the headline speedup at the largest memory size: $\mathbf{5.33}\times$ on Llama-3.1-8B, $\mathbf{5.65}\times$ on Mistral-7B, and $\mathbf{7.12}\times$ on Qwen2.5-7B. The Qwen result is the largest of the three because its more aggressive grouped-query attention — 4 KV heads against 8 in Llama and Mistral — and 28 transformer layers (vs. 32) cut per-token KV storage by more than half (roughly 56 KB vs. 128 KB), so the chunked-RoPE injection cost stays proportionally lower as N grows. Averaged across the four memory sizes, the KV-over-shuf ratio is $3.33\times$ on Llama, $3.57\times$ on Mistral, and $4.08\times$ on Qwen.

Chunked-RoPE also exceeds the *best-case* PI condition (*PI / prefix cache / fixed*) by $1.2\times$ to $3.8\times$ across the three models, with the largest advantage at $N{=}4096$ on Qwen ($3.83\times$). Even when PI gets full within-user cache reuse on the second request, the first request still pays the cold-cache prefill, while chunked-RoPE injects the composite KV directly on every request, including the first.

6.4 LoCoMo Benchmark: Long-Conversation Memory QA

The synthetic experiments above use short memory profiles ($\leq$144 tokens). To evaluate on realistic long-form conversational memory, we use the LoCoMo benchmark [3], which provides multi-session conversation histories (averaging $\sim$8K tokens) with ground-truth QA pairs across five categories of increasing reasoning difficulty: (1) single-hop factual, (2) multi-hop factual, (3) open-ended, (4) temporal reasoning, and (5) adversarial/unanswerable. We evaluate three conditions on all three models: *full context* (the entire conversation history in the prompt), *prompt injection* (extracted memory summaries prepended to the query), and *KV injection* (the same summaries pre-encoded as KV tensors).

6.5 LLM-as-Judge Semantic Evaluation

Because correct answers frequently use different phrasing than the reference (e.g., "the group had dinner at an Italian place" vs. "they went to an Italian restaurant"), token-overlap metrics underestimate all conditions equally, yielding low absolute scores even for full-context serving. To obtain a more meaningful quality signal, we employ an *independent LLM judge*.

We serve Gemma-2-9B-Instruct [8] via vLLM as a zero-shot semantic judge. For each of the 1,986 LoCoMo QA pairs $\times$ 3 conditions $\times$ 3 evaluated models (17,874 judgments total), the judge receives

the question, reference answer, and predicted answer and returns a binary correct/incorrect verdict. We use two category-specific prompts: for answerable questions (categories 1–4), the prompt instructs the judge that a refusal to answer is *incorrect*; for adversarial questions (category 5), the prompt instructs that a refusal is *correct*. This distinction prevents inflating scores for methods that refuse to answer answerable questions. The overall accuracy is computed over categories 1–4 only, since category 5 tests a distinct capability (knowing when *not* to answer).

Table 5 reports the results. On Llama-3.1-8B, KV injection achieves **67.2%** overall accuracy, surpassing both full context (61.6%) and prompt injection (25.6%). KV injection matches full context on single-hop (55.0% on both) and exceeds it on every other answerable category, with the largest gains on open-ended (+21.9 pp) and multi-hop (+8.1 pp) questions. On Mistral-7B, KV injection achieves 64.0% overall, a 2.1 pp improvement over full context (61.9%) driven by gains on multi-hop (+4.1 pp), open-ended (+15.6 pp), and temporal (+2.2 pp); single-hop is the only category where full context retains a small edge (59.2% vs. 54.6%). On Qwen2.5-7B, KV injection (66.6%) and full context (65.9%) are within 0.7 pp overall, with KV winning single-hop and temporal and full context winning multi-hop and open-ended. KV injection improves over prompt injection by 41.6 pp on Llama, 31.1 pp on Mistral, and 15.8 pp on Qwen, with the gap largest on the model where prompt injection struggles most. On category 5 (adversarial), prompt injection scores highest because it frequently refuses to answer—a behavior that is correct for unanswerable questions but detrimental for answerable ones, as reflected in its low categories 1–4 scores.

The result that KV injection matches or exceeds full-context quality while prompt injection does not may seem surprising, since both inject the same memory text. We attribute this to token budget effects: prompt injection consumes input context capacity with memory tokens, which can cause vLLM's scheduler to truncate or deprioritize portions of the input under memory pressure. KV injection avoids this by injecting memories outside the token stream.

6.6 QA latency

Table 5 reports end-to-end latency per question. On Llama, KV injection achieves a mean latency of 0.34 s—1.68× faster than prompt injection (0.57 s), 1.79× faster than Mem0 (0.61 s), and 5.56× faster than full context (1.89 s). On Mistral, KV injection achieves 0.34 s mean—1.21× faster than prompt injection (0.41 s), 1.59× faster than Mem0 (0.54 s), and 5.03× faster than full context (1.71 s). On Qwen, KV injection achieves 0.36 s mean—1.28× faster than prompt injection (0.46 s), 1.56× faster than Mem0 (0.56 s), and 4.06× faster than full context (1.46 s). KV injection has the lowest mean, P50, and P99 of any condition on all three models. The speedup over full context is substantial because LoCoMo conversations average ∼8K tokens, where the quadratic prefill cost dominates.

Comparison to Mem0. Mem0's throughput is largely insensitive to the "memory size" axis in our sweep because Mem0 retrieves top-k snippets (totaling roughly 200 tokens in our configuration) regardless of the underlying memory corpus; the effective prefill length is therefore bounded by the retrieval budget rather than by the memory itself. As a result, at small memory sizes (512 tokens) KV injection outperforms Mem0 by 1.5×–2.3× on all three models, while at larger memory sizes (2048–4096 tokens) Mem0 achieves comparable or slightly higher raw QPS. This throughput parity is deceptive: Mem0's top-k retrieval drops overall LoCoMo accuracy to 14.0% versus KV injection's 67.2% on Llama-3.1-8B-Instruct (Section 6.4, Table 5). Put differently, Mem0 trades 4.8× worse answer quality for throughput that is already matched or exceeded by KV injection at short contexts, and only modestly exceeded at long contexts where the quality gap is largest.

Cost relative to no-memory. We also report KV injection throughput relative to the no-memory baseline as an upper bound on what any memory-augmented system could achieve. KV injection retains 40%–88% of no-memory throughput depending on configuration, compared with 2%–17% for prompt injection. No memory-augmented condition matches NM in absolute QPS because decode still attends to the memory prefix; the relevant comparison is against PI, where KV injection recovers the bulk of the gap.

OOM behavior. For Llama-3.1-8B-Instruct and Mistral-7B-Instruct-v0.3, the $50\text{users} \times 4096\text{tokens}$ and $100\text{users} \times 2048\text{tokens}$ KV-injection configurations exceed the GPU memory budget once the pre-encoded memory store is resident alongside vLLM's paged cache (entries omitted from Tables 9 and 10). Qwen2.5-7B-Instruct has a smaller KV cache per token and runs all configurations to completion, reaching 74.97 QPS at $100\text{users} \times 2048\text{tokens}$ — a 9.83× improvement over prompt injection at the same configuration.

Table 6: LoCoMo QA latency comparison between GPU and CPU memory store (seconds).

Condition	Mean	P50	P99
Kv Injection	0.34	0.28	1.65
Kv Injection Cpu Offload	0.39	0.32	1.70

Summary. The throughput results confirm the analytic prediction from Section 3: eliminating $O(m^2)$ self-attention among memory tokens at prefill translates directly into near-linear throughput degradation with memory size, rather than the quadratic collapse exhibited by prompt injection. On all three models, KV injection delivers single-digit to double-digit speedups over PI across the full sweep, and pairs these throughput gains with the full-context answer quality documented in Section 6.4.

6.7 Scaling the Memory Store Beyond GPU HBM

The preceding experiments hold the authoritative KV cache in GPU HBM, which bounds the memory store's capacity to the device's high-bandwidth memory. Production deployments will need to serve far more users than fit in HBM at once. We therefore evaluate a scale-out variant of the memory store in which the authoritative KV resides in pinned host DRAM and is streamed to the GPU over PCIe on demand, asynchronously overlapped with the scatter-copy into vLLM's paged cache.

Implementation. The CPU-resident store allocates one page-locked host tensor per layer per user at ingest time. On every request, the connector issues per-layer `cudaMemcpyAsync` calls on a dedicated CUDA copy stream before entering the scatter-copy loop; each layer's scatter waits on the per-layer copy event via `wait_event`, so scatter for layer k can begin as soon as the bytes for layer k have landed on the device, without serializing against layers $k{+}1,...,L$. This yields a pipeline in which PCIe transfer overlaps with in-progress scatter compute.

Setup. We run the full LoCoMo QA suite (1,986 queries across 10 conversations) under two conditions on Llama-3.1-8B-Instruct: the default HBM-resident KV injection, and the PCIe-streamed variant described above. Hardware: one NVIDIA RTX PRO 6000 Blackwell with PCIe 5.0 $\times$16 (theoretical peak $\approx$63 GB/s per direction).

Results. Table 6 reports LoCoMo QA latency of GPU and CPU memory stores. Across all 1,986 transfers, the PCIe-streamed variant moves 4,913 GB of KV data at a mean achieved bandwidth of 56.12 GB/s (89.1% of PCIe 5.0 $\times$16 theoretical peak), with a mean raw transfer latency of 44.1 ms for an average payload of 2.47 GB per request. Crucially, the overlap ratio is 0.9996 (minimum 0.9994 across the ten conversations), meaning only $\approx$16 μs of transfer time falls on the critical path on average—virtually all PCIe cost is hidden behind scatter-copy compute. End-to-end, the scale-out variant adds 50 ms of mean latency per query (0.39 s vs 0.34 s, a 14.7% overhead), which we attribute to staging-buffer allocation and Python-level orchestration rather than PCIe transfer itself; the residual is bounded below by the non-transfer portion of the offload path and cannot be reduced by faster interconnects.

Task quality is identical under both conditions: F1 of 0.261 ± 0.308, exact-match of 0.082, with bitwise-matching per-category breakdowns. This confirms that the pinned-host $\rightarrow$ device copy path preserves KV tensors exactly, and that injecting PCIe-streamed memory produces the same attention outputs as HBM-resident injection.

Implication. Because the raw transfer saturates the bus and is almost entirely hidden, the memory store's effective capacity is no longer bounded by HBM: on a system with 1 TB of host DRAM, SALTMEM can hold $\approx 500\times$ more per-user KV than would fit in a single 96 GB HBM device, at a fixed $\approx 15\%$ latency overhead per query. This enables multi-tenant deployments in which the *per-user* memory working set can be much larger than any one device, while PCIe-bound scale-out is transparent to the attention computation.

7 Conclusion

We established an exact equivalence between two conditioning mechanisms in causal decoder transformers: prompt injection, in which a static context is processed through full prefill at every request, and KV

injection, in which the context's KV cache representations are precomputed once and inserted directly into the serving engine's paged cache. The equivalence (Theorem 3) holds for any causal transformer regardless of positional encoding, attention variant, or feed-forward architecture, and fails cleanly under three characterized conditions: bidirectional attention, position reassignment, and encoder-decoder cross-attention. Because equivalence is exact, the computational savings from KV injection—$\Theta(m^2)$ attention FLOPs per layer, per request, for a context of length m—come at zero quality cost.

We validated the equivalence empirically on three open-weight transformers and realized the compute savings as an end-to-end system integrated with vLLM. The resulting prototype, SALTMEM, delivers up to 12.85× throughput improvement over prompt injection in multi-user serving and matches full-context QA accuracy on LOCOMO at 5.56× lower latency, without modifying vLLM's internals.

The equivalence generalizes beyond user memory. Any deployment pattern that repeatedly prepends a static context to varying queries is an instance: retrieval-augmented generation with a stable corpus, cached system prompts, fixed few-shot exemplars, persistent tool descriptions. For all of these, the KV representations of the static context can be computed once and reused indefinitely, with the same formal guarantees we establish here. As per-request static context grows relative to per-request query—the dominant scaling trend in deployed LLM applications—the gap between prompt injection and KV injection widens without bound. We expect attention-layer injection to become the default primitive for conditioning LLMs on static context, and offer this paper and the accompanying open-source implementation as a starting point for that transition.

References

[1] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.

[2] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[3] Jiarui Zhang, Jonathan Raiman, and Sathish Reddy Indurthi. LOCOMO: Long-context conversation memorization benchmark. *arXiv preprint arXiv:2402.17753*, 2024.

[4] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

[5] Mem0. Mem0: The memory layer for AI agents. `https://github.com/mem0ai/mem0`, 2024.

[6] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

[7] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

[8] Gemma Team. Gemma: Open models based on Gemini research and technology. *arXiv preprint arXiv:2403.08295*, 2024.

[9] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. *arXiv preprint arXiv:2312.07104*, 2023.

[10] Yuhan Yao, Hao Zhang, and Ion Stoica. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. *arXiv preprint arXiv:2405.16444*, 2024.

[11] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[12] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

A Extended Theoretical Results

A.1 Complexity Separation

Theorem 8 (Complexity Separation)**.** *For a single attention layer with n_h heads of dimension d, memory length m, and query length q:*

1. ***Prompt injection*** *requires $\Theta\big((m+q)^2 \cdot n_h d\big)$ attention FLOPs for the prefill pass.*
2. ***KV injection*** *requires $\Theta\big(q \cdot (m+q) \cdot n_h d\big)$ attention FLOPs at serving time (excluding the one-time encoding cost).*
3. *The per-request FLOP savings is:*

$$\Delta = \Theta(m^2 \cdot n_h d) \quad .$$

The total savings across L layers and R requests is $\Theta(R \cdot L \cdot m^2 \cdot n_h d)$, while the one-time encoding cost is $\Theta(L \cdot m^2 \cdot n_h d)$—amortized to zero as $R \to \infty$.

Proof. Under prompt injection, each of the $(m+q)$ tokens computes attention over all preceding tokens, yielding total attention FLOPs of $\sum_{i=1}^{m+q} i \cdot n_h d = \Theta((m+q)^2 \cdot n_h d)$ per layer.

Under KV injection, only the q query tokens undergo prefill attention. Token at query position i (global position $m+i$) attends to $m+i$ KV vectors, giving $\sum_{i=1}^{q} (m+i) \cdot n_h d = \Theta(q \cdot (m+q) \cdot n_h d)$.

The difference is:

$$\Delta = m^2,$$

multiplied by $n_h d$.

For amortization: the one-time encoding cost of processing M alone is $\Theta(m^2 \cdot n_h d)$ per layer (same as prompt injection for M). After R requests, the encoding cost per request is $\Theta(m^2 \cdot n_h d / R) \to 0$. □

A.2 Architectural Boundaries

Theorem 3 relies on two structural properties: causal masking and consistent position assignment. We now characterize when these conditions fail.

Proposition 9 (Non-Equivalence under Bidirectional Attention)**.** *If the attention mask at any layer ℓ is bidirectional (i.e., position $j \leq m$ can attend to positions $j' > m$), then KV injection and prompt injection produce different hidden states at memory positions in general. Formally, there exist inputs M, Q such that:*

$$\tilde{\mathbf{h}}_j^{(\ell)} \neq \mathbf{h}_j^{(\ell)}\big|_{\mathrm{PI}} \quad \textit{for some } j \leq m, \ell \geq 1.$$

Proof. Under bidirectional attention at layer ℓ, the hidden state $\mathbf{h}_j^{(\ell)}\big|_{\mathrm{PI}}$ for a memory position $j \leq m$ depends on $\{\mathbf{h}_k^{(\ell-1)}\}_{k=1}^{m+q}$, including query positions. Under KV injection, $\tilde{\mathbf{h}}_j^{(\ell)}$ is computed from $\{\tilde{\mathbf{h}}_k^{(\ell-1)}\}_{k=1}^{m}$ only, since the query is absent during encoding. Because $\mathrm{Attn}^{(\ell)}$ is in general not invariant to the addition of extra key-value pairs, the outputs differ. □

Scope. Proposition 9 rules out encoder-only architectures (BERT, RoBERTa) and the encoder component of encoder-decoder models (T5, BART). It does *not* rule out the decoder component of encoder-decoder models, where causal masking is applied—though cross-attention to encoder states introduces a separate dependency (see Proposition 13 below).

Proposition 10 (Non-Equivalence under Position Reassignment)**.** *Suppose memory tokens are encoded at positions $0,1,\ldots,m-1$ but injected at positions $p_1,\ldots,p_m$ with $p_j \neq j-1$ for some j. For any position encoding f satisfying $f(x,p) \neq f(x,p')$ when $p \neq p'$ (including RoPE and non-degenerate learned embeddings), KV injection produces different attention scores than prompt injection.*

Proof. Consider RoPE. The pre-computed key at position j has the form $\tilde{\mathbf{k}}_j = f_{\mathrm{RoPE}}(\mathbf{W}_K \mathbf{h}_j, j-1)$. If this key is injected at position $p_j \neq j-1$, the attention score between query at position i and this key encodes a relative distance of $(i-(j-1))$, whereas prompt injection at position p_j would encode a relative distance of $(i-p_j) \neq (i-(j-1))$. The resulting softmax weights—and hence attention outputs—differ. □

Note: shift-invariant relative encodings (ALiBi being the canonical example) preserve equivalence under contiguous shifts.

Implication. Our system prepends memory at positions $0,...,m-1$ and appends the query at positions $m,...,m+q-1$, matching prompt injection exactly. Proposition 10 shows that extending KV injection to support *arbitrary-position insertion* (e.g., interleaving memory within the query) would require storing keys *before* positional encoding application and re-rotating at serving time.

Proposition 11 (Equivalence Extends to Grouped-Query and Multi-Query Attention)**.** *Let $\mathcal{T}$ use grouped-query attention (GQA) with n_q query heads and $n_k < n_q$ KV heads, or multi-query attention (MQA) with $n_k = 1$. Theorem 3 holds without modification.*

Proof. The proof of Theorem 3 depends on two properties: (1) the KV pairs at memory positions are functions only of memory-position hidden states (by causal masking), and (2) the hidden-state computation is deterministic. Both hold regardless of the number of KV heads: GQA and MQA change only how query heads are grouped to share KV heads, not the causal structure. The inductive argument applies identically. □

Proposition 12 (Equivalence Extends to Mixture-of-Experts)**.** *Let $\mathcal{T}$ be a causal decoder transformer with Mixture-of-Experts (MoE) feed-forward layers, where the expert routing function $g(\mathbf{h}_i^{(\ell)})$ selects a subset of experts for each position based on its hidden state. Theorem 3 holds without modification.*

Proof. By the inductive argument of Theorem 3, the hidden state $\mathbf{h}_j^{(\ell-1)}$ at each memory position $j \leq m$ is identical under KV injection and prompt injection. Since the routing function g is a deterministic function of $\mathbf{h}_j^{(\ell-1)}$, the same experts are selected. The MoE layer output at position j is therefore identical, and the induction proceeds unchanged. □

Proposition 13 (Non-Equivalence with Encoder-Decoder Cross-Attention)**.** *In encoder-decoder architectures where the decoder's cross-attention layers attend to encoder hidden states, KV injection of* encoder *representations is not equivalent to prompt injection if the encoder uses bidirectional attention (per Proposition 9). However, KV injection of* decoder-side *context (e.g., conversation history processed by the decoder) remains exact under causal masking.*

Summary. Table 7 summarizes the architectural compatibility of KV injection.

Table 7: Architectural compatibility of KV injection. "Exact" means Theorem 3 applies; "N/A" means the architecture does not use the relevant component.

Architecture	KV Injection Equivalence	Key Condition
Decoder-only, RoPE (Llama, Mistral, Qwen)	Exact	Causal mask
Decoder-only, ALiBi (Bloom, MPT)	Exact	Causal mask
Decoder-only, learned pos. emb. (GPT-2)	Exact	Causal mask
Decoder-only, MoE (Mixtral, DeepSeek)	Exact	Causal mask
GQA / MQA variants	Exact	Causal mask
Encoder-only (BERT, RoBERTa)	**Not equivalent**	Bidirectional attn
Encoder-decoder, encoder side (T5, BART)	**Not equivalent**	Bidirectional attn
Encoder-decoder, decoder side (T5, BART)	Exact	Causal mask
Arbitrary-position insertion	**Not equivalent**	Position mismatch

B Cost Implications

Production LLM memory systems are not deployed as naive prompt injection; they use retrieval-augmented architectures such as Mem0 that extract structured memory from conversations, embed it, and retrieve top-k snippets at query time. A fair cost analysis must compare KV injection against these systems on both axes where they differ: *serving-time cost* (per request at inference) and *ingestion-time cost* (the one-time cost of preparing a user's memory for retrieval). We find that KV injection improves on both, but the ingestion-side gap is the more striking one.

Serving-time cost. Section C.2 reports end-to-end QA latency on LOCOMO for all four conditions. KV injection is **1.5–1.8× faster** than Mem0 at serving time on all three models (mean latency 0.34–0.36 s vs. 0.54–0.61 s), with a comparable P99 gap. The mechanism is mechanical: Mem0 inserts a retrieval step—embedding the query, searching a vector database, and re-tokenizing the top-k snippets—before the model's prefill can begin. KV injection skips all of this: the scheduler identifies the user, scatter-copies the pre-encoded KV into allocated paged blocks, and prefill runs directly over the query. The serving-cost advantage is modest in absolute terms (each request is already sub-second on all methods), but it compounds with the quality gap from Section 6.4: at comparable latency, Mem0 answers 14.0% of LOCOMO questions correctly on Llama versus KV injection's 67.2%—a 4.8× quality gap on a task where both systems have access to the same underlying conversations.

Ingestion-time cost. The asymmetry is even sharper on the ingestion side. For each of the three models, we measured the wall-clock time for Mem0 to ingest the 10 LOCOMO conversations into its memory database—running the same backbone model (served via vLLM) as the fact-extraction LLM:

- **Llama-3.1-8B-Instruct**: 5,321 s total (**532 s per conversation**, 6.03 s per extracted memory; 883 memories extracted)
- **Mistral-7B-Instruct-v0.3**: 4,332 s total (**433 s per conversation**, 4.33 s per extracted memory; 1,000 memories extracted)
- **Qwen2.5-7B-Instruct**: 3,218 s total (**322 s per conversation**, 3.23 s per extracted memory; 995 memories extracted)

On average, **Mem0 takes 7.2 minutes of GPU time to ingest a single user's conversation**, because its pipeline makes one LLM call per candidate fact to extract, consolidate, and deduplicate memories against prior state. The extraction LLM runs on the same Blackwell hardware as the serving model; the cost is not network latency but genuine forward-pass compute, multiplied by hundreds of sequential calls per user.

KV injection's ingestion pipeline has a different shape. For a user with m memory tokens, encoding consists of *a single forward pass of length m through the serving model*, producing the KV tensors that are persisted to disk. On the same Blackwell hardware running Llama-3.1-8B-Instruct, a $2{,}048$-token forward pass completes in a fraction of a second (the TTFT figures in Section C.1 give an upper bound: the combined memory+query prefill for $m+q=2{,}048$ tokens takes well under one second end-to-end, and the memory-only encoding pass is a strict subset of that work). Compared to Mem0's measured 532 s per user on Llama, KV injection's per-user encoding cost is smaller by **three orders of magnitude**—and unlike Mem0, it does not require a separate extraction prompt, a separate vector index, or sequential calls that scale with the number of extracted facts.[2]

Why the ingestion gap matters in deployment. In production, memory ingestion happens every time a user's conversation grows, a new document enters their corpus, or a scheduled re-indexing runs. A 7-minute-per-user ingestion latency means Mem0 cannot keep up with real-time conversation streams at scale without a large standing fleet of extraction GPUs—a fleet that sits idle between ingestion spikes but must be provisioned for peak load. KV injection's sub-second per-user encoding makes real-time ingestion tractable on the same fleet that handles serving: the encoding pass is short enough to be scheduled as a low-priority background task on any GPU with spare capacity, or triggered on-demand the first time a user with new memory issues a query (with a one-time added latency well below the serving latency of a single Mem0 query).

Storage footprint. The one dimension on which Mem0 wins is storage. Mem0 stores extracted facts as short text strings plus dense embeddings—typically kilobytes per memory. KV injection stores full KV tensors—for Llama-3.1-8B-Instruct with 32 layers, 8 KV heads, 128 head dim, and bfloat16, a single memory token costs $32 \times 2 \times 8 \times 128 \times 2 = 131{,}072$ bytes, so a $2{,}048$-token memory is 256 MB per user. At 10,000 users, this is 2.5 TB of KV-cache storage—manageable on modern GPU servers with NVMe tiering, but a real engineering constraint. This cost is a direct consequence of the exact-equivalence property: KV injection stores everything the model needs to reproduce full-context attention, whereas Mem0's compact storage is the compression-step that also causes its 4.8× quality degradation.

[2] We report KV encoding as a theoretical bound derived from forward-pass latency rather than a direct measurement; a precise measurement is a trivial instrumentation change and would sharpen the comparison further. The order-of-magnitude gap is robust to a factor of 2–3 in either direction.

Table 8: Time-to-first-token (ms) vs. memory size across three models. **Mem.**: number of memory tokens. **PI**: prompt injection. **KV**: KV-cache injection (SALTMEM). **Spd.**: speedup of KV over PI. KV injection eliminates the prompt-prefill cost; speedup grows with memory size.

Llama-3.1-8B-Instruct

Mem.	PI	KV	Spd.
0	577	—	—
512	597	580	1.03×
2,048	673	587	1.15×
8,192	1,035	620	1.67×
16,384	1,664	664	2.51×
32,768	3,300	753	**4.39**×

Mistral-7B-Instruct-v0.3

Mem.	PI	KV	Spd.
0	548	—	—
512	568	550	1.03×
2,048	645	558	1.15×
8,192	1,005	591	1.70×
16,384	1,638	635	**2.58**×

Qwen2.5-7B-Instruct

Mem.	PI	KV	Spd.
0	564	—	—
512	578	564	1.03×
2,048	641	566	1.13×
8,192	953	583	1.63×
16,384	1,495	605	**2.47**×

Composite picture. Consolidating across the three cost axes: at serving time, KV injection is 1.5–1.8× faster than Mem0 *and* 4.8× higher quality (on Llama). At ingestion time, KV injection is roughly three orders of magnitude faster. The price is approximately $100\times$ more storage per user. For deployments where (a) ingestion latency is on the critical path—real-time personalization, agentic memory updates, streaming conversation ingestion—or (b) quality at comparable serving cost is the binding constraint, KV injection is the more economical choice. Mem0 remains attractive when storage is the binding constraint and the application can tolerate minutes-scale ingestion latency and substantially reduced answer accuracy.

B.1 Relationship to Prefix Caching

Prefix caching [9, 4] is the most frequently-raised alternative to KV injection and is easy to mistake for the same mechanism. Both store pre-computed KV tensors and skip prefill for the cached prefix. The distinction is the *lookup key*: prefix caching is content-addressed, while KV injection is user-addressed.

Prefix caching hashes the token prefix of an incoming request against recently-observed prefixes. Matches are opportunistic—they require exact token agreement with a prefix the engine has not yet evicted—and the application layer is not involved. This works well when many requests share the same tokens (e.g., a global system prompt) but thrashes when prefixes are per-user: no two users share a memory prefix, and a user's cached prefix evicted during an idle period cold-starts the next request. KV injection, by contrast, is invoked by the application with an explicit `user_id`. The memory's KV representation is pre-computed once, persisted, and injected deterministically on every request from that user, regardless of what other traffic the engine has seen.

The two techniques are complementary. A production stack has at least three layers of prefix content: a global system prompt (same for all users, best served by prefix caching), per-user memory (distinct per user, best served by KV injection), and the per-request query (not cached). The layers compose cleanly—prefix caching at positions $0,...,s-1$; KV injection at $s,...,s+m-1$; prefill at $s+m,...$. Our throughput results disable prefix caching to isolate KV injection's contribution, but the two are mechanically independent: prefix caching operates on vLLM's hash of the request's tokens, and our connector's externally-matched-tokens signal does not interfere with that hash. Both techniques rest on the same underlying fact—that a static prefix's KV is a deterministic function of the prefix and is therefore cacheable—which is the exact equivalence we formalize in Section 3. Prefix caching exploits this at the granularity of tokens with opportunistic lookup; KV injection exploits it at the granularity of named entities with explicit storage.

C Extended Evaluation

C.1 Single-Request Latency (TTFT)

We measure time-to-first-token (TTFT) as memory size grows from 0 to 32,768 tokens (Llama) and 0 to 16,384 tokens (Mistral, Qwen). Each request generates exactly 50 output tokens with a fixed query ($\sim$40 tokens). Table 8 reports all ttft latencies for all methods across three models.

The results confirm the theoretical analysis across all three models. Prompt injection TTFT grows superlinearly with memory size, consistent with quadratic prefill cost. KV injection TTFT increases only marginally, dominated by the fixed decode overhead with minimal scatter-copy cost. All three models

Table 9: End-to-end serving throughput (QPS) on **Llama-3.1-8B-Instruct**. NM = no memory, PI = prompt injection, Mem0 = retrieval baseline, KV = our KV injection. Speedups are relative to PI (KV/PI) and Mem0 (KV/Mem0). "—" denotes out-of-memory.

Configuration		Throughput (QPS)				KV Speedup	
Users	Mem. tok.	NM	PI	Mem0	KV	vs. PI	vs. Mem0
10	512	30.37	15.85	14.68	26.17	1.65×	1.78×
10	1024	30.39	10.57	22.69	24.50	2.32×	1.08×
10	2048	30.39	6.20	25.00	22.05	3.56×	0.88×
10	4096	30.39	3.27	24.99	18.31	5.60×	0.73×
25	512	65.85	21.33	31.70	48.45	2.27×	1.53×
25	1024	65.11	12.59	43.15	43.20	3.43×	1.00×
25	2048	65.18	6.74	44.39	35.52	5.27×	0.80×
25	4096	65.99	3.37	44.48	26.15	7.77×	0.59×
50	512	108.39	23.87	39.09	69.64	2.92×	1.78×
50	1024	108.78	13.38	55.38	59.27	4.43×	1.07×
50	2048	107.12	6.90	58.24	46.11	6.68×	0.79×
50	4096	108.54	3.30	58.44	—	—	—
100	512	143.39	25.23	42.67	82.50	3.27×	1.93×
100	1024	143.24	13.61	60.89	60.83	4.47×	1.00×
100	2048	143.47	6.68	65.51	—	—	—

exhibit nearly identical scaling behavior, demonstrating that the speedup is a property of the attention mechanism, not a model-specific artifact.

C.2 Serving Throughput

We measure end-to-end serving throughput on an NVIDIA RTX PRO 6000 Blackwell (96 GB) under four conditions: *no-memory* (NM), where requests carry only the system prompt and query; *prompt injection* (PI), where memory tokens are prepended as text and prefilled in full; *Mem0* [1], a retrieval-augmented baseline that selects top-k memory snippets via dense retrieval and injects them as prompt text; and *KV injection* (ours), which scatter-copies pre-encoded memory KV tensors into vLLM's paged cache. All four conditions use vLLM 0.16.0 with FlashInfer attention, bf16 weights, and identical generation settings (1000 decode tokens per request, greedy). Each configuration is swept across user counts $\{10,25,50,100\}$ and memory sizes $\{512,1024,2048,4096\}$ tokens, with one request per user per batch.

Table 9, Table 10, and Table 11 report absolute throughput (requests per second, QPS) for all three models. Across configurations, KV injection delivers **1.65×–12.85× higher throughput than prompt injection**, with the gap widening as memory size grows. The largest per-configuration speedup we observe is 12.85× on Qwen2.5-7B-Instruct at 50 concurrent users with 4096-token memories; Mistral-7B-Instruct-v0.3 peaks at 8.21× (25 users, 4096 tokens) and Llama-3.1-8B-Instruct at 7.77× (25 users, 4096 tokens). The scaling pattern is consistent across all three models: doubling the memory size roughly halves PI's throughput (because PI pays $O((m+q)^2)$ attention at prefill over the combined memory and query), whereas KV injection's throughput falls much more gently because attention over the memory prefix is $O(q \cdot m)$ rather than $O((m+q)^2)$.

C.3 Scaling the Memory Store Beyond GPU HBM

D Related Work

LLM memory systems. Mem0 [5] and Zep provide retrieval-augmented memory using vector databases and knowledge graphs respectively. MemGPT/Letta [2] implements a virtual memory hierarchy inspired by operating systems. All of these systems inject retrieved memories as prompt text. Our work is complementary: we can serve as the *injection backend* for any of these systems, replacing their prompt injection step with KV injection.

Table 10: End-to-end serving throughput (QPS) on **Mistral-7B-Instruct-v0.3**. Columns as in Table 9.

Configuration		Throughput (QPS)				KV Speedup	
Users	Mem. tok.	NM	PI	Mem0	KV	vs. PI	vs. Mem0
10	512	30.31	15.85	15.12	28.49	1.80×	1.88×
10	1024	30.37	10.55	23.91	26.47	2.51×	1.11×
10	2048	30.37	6.17	26.83	23.79	3.86×	0.89×
10	4096	30.37	3.26	26.79	19.36	5.94×	0.72×
25	512	65.92	21.26	33.47	53.54	2.52×	1.60×
25	1024	65.70	12.53	47.20	47.58	3.80×	1.01×
25	2048	66.06	6.72	48.67	38.47	5.73×	0.79×
25	4096	65.22	3.36	48.15	27.55	8.21×	0.57×
50	512	108.47	23.84	41.17	78.74	3.30×	1.91×
50	1024	108.25	13.35	60.95	66.00	4.94×	1.08×
50	2048	108.80	6.89	65.32	50.40	7.31×	0.77×
50	4096	107.52	3.29	64.42	—	—	—
100	512	143.48	25.18	45.17	97.61	3.88×	2.16×
100	1024	144.39	13.60	67.61	80.41	5.91×	1.19×
100	2048	143.30	6.68	59.69	—	—	—

Table 11: End-to-end serving throughput (QPS) on **Qwen2.5-7B-Instruct**. Columns as in Table 9. All configurations complete without OOM on this model.

Configuration		Throughput (QPS)				KV Speedup	
Users	Mem. tok.	NM	PI	Mem0	KV	vs. PI	vs. Mem0
10	512	33.35	17.50	15.53	29.22	1.67×	1.88×
10	1024	33.37	11.74	24.07	28.23	2.41×	1.17×
10	2048	33.37	6.89	26.93	26.62	3.86×	0.99×
10	4096	33.37	3.66	26.71	23.90	6.54×	0.89×
25	512	77.12	24.14	34.81	58.31	2.42×	1.68×
25	1024	76.59	14.12	48.31	54.47	3.86×	1.13×
25	2048	76.04	7.57	49.07	47.88	6.33×	0.98×
25	4096	77.17	3.79	48.82	39.17	10.35×	0.80×
50	512	130.41	27.17	43.06	84.86	3.12×	1.97×
50	1024	131.75	15.02	60.20	75.85	5.05×	1.26×
50	2048	132.83	7.72	64.92	64.42	8.34×	0.99×
50	4096	132.39	3.76	64.50	48.34	12.85×	0.75×
100	512	186.11	28.80	47.04	106.94	3.71×	2.27×
100	1024	188.95	15.34	68.19	92.94	6.06×	1.36×
100	2048	187.68	7.62	63.09	74.97	9.83×	1.19×

KV cache optimization. PagedAttention [4] introduced paged memory management for KV caches and is the substrate on which our system is built. CacheBlend [10] enables non-prefix KV reuse by *selectively recomputing* attention for a small number of tokens to restore positional coherence across concatenated cache segments—an approximation that trades quality for reuse flexibility. Our approach is the dual: we preserve the prefix structure (memory at positions $0,...,m-1$) and, by the argument of Proposition 10, achieve exact equivalence without any recomputation. CacheBlend is appropriate when position layout is unconstrained; our method is appropriate when positions can be fixed at encoding time, which is the typical case for user memory, cached system prompts, and RAG corpora. A parallel line of work compresses the stored KV cache via quantization—KVQuant [11] quantizes to 3 bits per channel with per-channel and pre-RoPE strategies, achieving near-lossless long-context inference. This direction is orthogonal and composable with KV injection: quantization reduces the *representation cost* of stored KV vectors, while KV injection eliminates the *computation cost* of re-deriving them at prefill. The two techniques can be combined without interference, since the KV-injection theorem holds for any deterministic function mapping tokens to KV representations, including a quantized one.

Table 12: Per-conversation PCIe transfer statistics for the CPU-offloaded memory store on LoCoMo (Llama-3.1-8B-Instruct, RTX PRO 6000 Blackwell, PCIe 5.0 $\times$16). Each row summarizes all H2D transfers issued during QA evaluation of one conversation. Overlap ratio is the fraction of transfer time hidden behind scatter-copy compute.

Conv. ID	#Transfers	Bytes (GB)	Latency (ms)	Bandwidth (GB/s)	Overlap
conv-26	199	380.5	34.1	56.02	0.9996
conv-30	105	156.5	26.7	55.84	0.9994
conv-41	193	552.5	50.9	56.21	0.9997
conv-42	260	644.2	44.1	56.14	0.9997
conv-43	242	672.9	49.5	56.19	0.9997
conv-44	158	432.2	48.7	56.17	0.9997
conv-47	190	504.2	47.2	56.17	0.9997
conv-48	239	608.3	45.3	56.14	0.9997
conv-49	196	416.9	37.9	56.06	0.9996
conv-50	204	545.1	47.6	56.16	0.9997
Total / Avg.	**1,986**	**4,913.4**	**44.1**	**56.12**	**0.9996**

Prefix caching. vLLM's built-in prefix caching [9] reuses KV blocks when multiple requests share a common prompt prefix. This helps when many requests share the *same* system prompt, but does not help with *per-user* memory, which differs across users. KV injection complements prefix caching: shared system prompt tokens can be prefix-cached while per-user memory tokens are KV-injected.

Retrieval-augmented generation. RAG systems [12] retrieve documents and inject them into prompts. The same $O(m^2)$ prefill overhead applies to retrieved documents. KV injection could be extended to pre-encode frequently-retrieved documents, though the static-content assumption is weaker for RAG (retrieved documents change per query).